

DD2459 Software Reliability 2021
Take-home Exam
Monday 15 March, 15.00 – Wednesday 17 March, 12.00
(Version 2)

Instructions.

Please read the following instructions carefully.

- (1) Clearly mark at the top of each sheet you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) Your work should be uploaded in one .pdf file to the course Assignments page on Canvas no later than Wednesday 17 March 2021 at 12.00 midday. After this time, it will be marked as late and points will be subtracted.
- (4) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm! You can also e-mail the examiner at karlm@kth.se.
- (5) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates to this document.
- (6) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page. (d) In the case that cheating is suspected, disciplinary action will be taken.
- (7) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

Please turn over.

NOTE: For a full score of 80 points you should answer all 4 questions below.

Question 1. (Total 15 points)

Write appropriate pre- and post-conditions for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid under-specification, i.e. write all the constraints necessary for each condition and do not omit any relevant constraints.

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.

You do not have to provide Java code for any of the operations, only JML.

- (i) **(5 points)** A *factorial function*:

```
int Factorial(int x)
```

Input: One positive integer parameter x .

Output: The integer result which should be $x!$

Important Note: Since most JML tools cannot handle recursive JML models you should not use a recursive solution such as: `\result == x * Factorial(x-1)`

- (ii) **(5 points)** A *Fibonacci function*:

```
int Fibb(int n)
```

Input: One positive integer parameter n .

Output: The n th number in the Fibonacci series.

Important Note: Since most JML tools cannot handle recursive JML models you should not use a recursive solution such as: `\result == Fibb(n-1) + Fibb(n-2)`

- (iii) **(5 points)** A *Greatest Common Divisor (GCD) function*:

```
int GCD(int x, int y)
```

Input: Two integer parameters x , y , not both 0.

Output: The greatest common divisor (GCD) of x and y .

Question 2. (Total 5 points)

- (i) **(3 points)** Write down one metamorphic equation that should be satisfied by the arithmetic average function:

`float average(float[] distribution)`

Input: A one-dimensional array `distribution` of `float` (the data set).

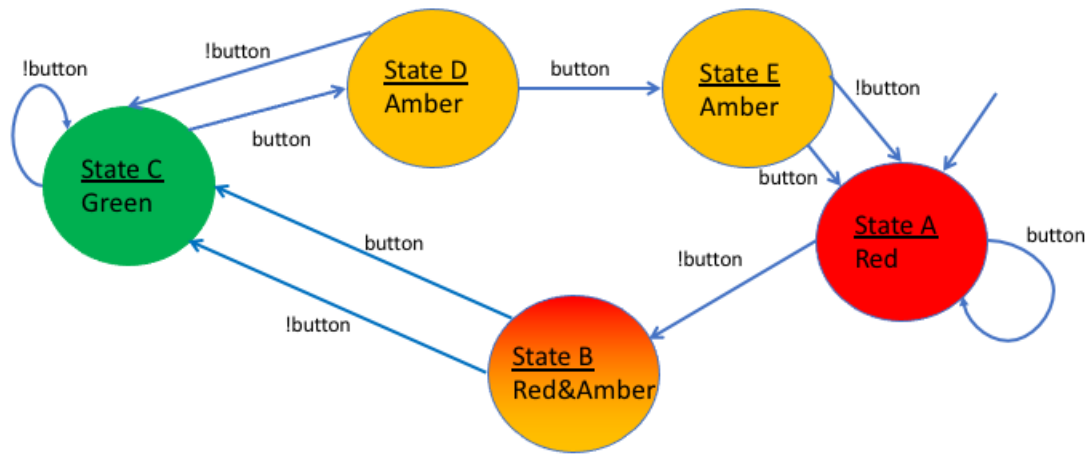
Output: The numerical average of the values in `distribution`.

Clearly define the data transformation used on the input in your equation.

- (ii) **(2 points)** Starting from any concrete seed test case (i.e. input array values), write down a second test case using your data transformation defined in your answer to part (i).

Question 3. (Total 21 points) The diagram below shows a five state *finite state machine* (FSM) model of a *pedestrian road crossing at a traffic light*. The traffic light can give one of four different output values (light colours) as signals to vehicles. The system has a single Boolean input variable *button* (a pedestrian request to cross), which can be true or false at any time. The transitions for each state are shown in the diagram.

Pedestrian Road Crossing Finite State Machine



(a) **(3 points)** Write down a set of linear temporal logic (LTL) trap properties that would achieve *full node coverage* (NC) when generating test cases as counterexamples to the properties. You do not need to eliminate redundant requirements.

(b) **(5 points)** Write down a set of linear temporal logic (LTL) trap properties that would achieve *full edge coverage* (EC) when generating test cases as counterexamples to the properties. You do not need to eliminate redundant requirements.

(c) **(8 points)** Write down a set of linear temporal logic (LTL) trap properties that would achieve *full edge-pair coverage* (EPC = EC²) when generating test cases as counterexamples to the properties. You do not need to eliminate redundant requirements.

(d) **(5 points)** An FSM A (such as the Pedestrian Road Crossing FSM model above) can be said to be *fair* with respect to any state *s* of A if the LTL formula

$$G(F(\text{state} = s))$$

is a *valid LTL formula* for A. Recall that an LTL formula *f* is *valid* for A if, and only if, there is no counterexample $i = (i_1, i_2, i_3, \dots)$ (an infinite input sequence for A) that makes *f* *false*.

(d.i) Explain *in words*, precisely what the above fairness formula means in terms of state *s*.

(d.ii) Discuss whether the Pedestrian Road Crossing FSM above is fair for each of its 5 states. Motivate your answer.

(d.iii) Is it possible to *demonstrate* that a system is unfair (using the definition above) by software testing? Motivate your answer and discuss any *alternative approaches* to the problem.

Question 4. (Total 39 points) Consider the following ten lines of pseudo-code:

```
1.   result = 0;
2.   while (y > 0) do {
3.       z = x;
4.       while (z > 0) do {
5.           result = result + 1;
6.           z = z - 1;
7.       }
8.       y = y - 1;
9.   }
10.  return result;
```

You can assume that x :integer and y :integer are the only integer input variables to the above pseudo-code.

- (a) **(2 points)** Draw a condensation graph for the above pseudo-code. Clearly number all the nodes in your graph.
- (b) **(10 points)** Show that for your condensation graph in answer to part (a) that node coverage (NC) is the same as edge coverage (EC), i.e. let TS be *any* test suite for your condensation graph, then TS satisfies 100% node coverage if, and only if, TS satisfies 100% edge coverage.
- (c) **(5 points)** Use your reasoning for part (b) to define a general graph theoretic property P of condensation graphs such that: for any condensation graph G satisfying P, NC and EC always give the same coverage of G. Show that there exists at least one condensation graph G that does not satisfy P, and for this graph G, NC does not give the same coverage as EC. Hence conclude that in general NC is not the same as EC.
- (d) **(11 points)** For the condensation graph of part (a), write down a control flow test requirement tr such that a test case tc satisfies your test requirement tr if, and only if, tc executes the body of each while loop exactly once? (**Clarification:** the two while loop bodies are the code lines contained in parentheses $\{\}$ which start at lines 3 and 5 respectively in the pseudo-code.) How many test cases exist for this glass-box test requirement? Carefully explain your reasoning.
- (e) **(6 points)** For the condensation graph of part (a), give as many edge pair requirements as you can, *each* of which demonstrates that 100% edge pair coverage (E^2C) gives greater coverage than 100% edge coverage (EC) and 100% node coverage (NC). Clearly explain and motivate each answer, and discuss any redundancy among them.
- (f) **(5 points)** Give a JML specification (requires, ensures) that accurately describes the black-box functional behaviour of your condensation graph. Using this specification, give test cases and predicted outputs for each of your edge pair requirements in part (e).